

30. Базовые типы, конструкторы и выражения типа. Представление выражения типа дагом. Эквивалентность выражений типа.

Введение

Любой элемент/блок данных имеет свой тип, определяющий структуру данных, способ представления в памяти, допустимые методы обработки и самое главное – семантику, т. е. смысл самих данных.

Тип данных задается *выражением типа*. Такие выражения получаются применением операторов, называемых *конструкторами типов*, к операндам, представляющим собой элементарные, *базовые* типы, либо другие выражения типа. Кроме того, любое выражение типа можно снабдить именем (мы будем говорить о нем как об *имени типа*). Удобно также ввести в рассмотрение *переменные типа* – переменные, значениями которых являются выражения типа.

Базовые типы, конструкторы и выражения типа

Базовые типы — это элементарные типы данных (int, float, char, boolean), предоставляемые языком программирования.

Примеры: int, float, char, boolean, void

Назначение: задать элементарные единицы хранения и операций (сложение чисел, сравнение символов).

Конструкторы типов — это правила создания сложных типов из базовых (массивы, указатели, структуры).

Примеры:

Массив (Array): array[10] of int — тип "массив из 10 целых чисел".

Структура (Record/Struct): struct { int x; char c; } — тип, объединяющий несколько полей.

Функция (Function): (int, int) -> bool — тип "функция, принимающая два int и возвращающая bool".

Выражение типа — это формальное описание типа с использованием базовых типов и конструкторов.

Базовые типы (без внутренней структуры): числовые, байтовый и битовый типы и т.п. В примерах в качестве базовых рассматриваются типы `int`, `float`, `char` и `bool`. К ним добавляются два специальных типа – `type_error`, сигнализирующий об ошибке типов, а также `void`, указывающий на отсутствие возвращаемого значения и тем самым позволяющий анализировать инструкции и процедуры с точки зрения наличия ошибок типов.

Обозначения базовых типов будем называть *константами типа*. Мы будем использовать пять конструкторов типов – *массив*, *произведение*, *структура*, *указатель* и *функция*.

Определение выражения типа

1. Константа типа, имя типа и переменная типа являются выражениями типа.
2. Результат применения конструктора типов к выражениям типа является выражением типа. Более точно:
 - 1) *Массивы*. Если T – выражение типа и N – натуральное число, то `array( $N$ ,  $T$ )` является выражением типа, задающим тип массива длины N с элементами типа T и индексами от 0 до $N-1$.
 - 2) *Произведения*. Если T_1, T_2 – выражения типа, то их декартово произведение $T_1 \times T_2$ является выражением типа.
 - 3) *Структуры*. Если $T_1, \dots, T_k$ – выражения типа, $name_1, \dots, name_k$ – их имена, то `struct( $(name_1 \times T_1) \times \dots \times (name_k \times T_k)$ )` является выражением типа, задающим тип структуры с полями $T_1, \dots, T_k$, имеющими имена $name_1, \dots, name_k$ соответственно. Тип записи всегда имеет имя, которое задается при объявлении этого типа.
 - 4) *Указатели*. Если T – выражение типа, то `ptr( $T$ )` является выражением типа, задающим тип «указатель на объект типа T ».
 - 5) *Функции*. Если T_1, T_2 – выражения типа, то $T_1 \rightarrow T_2$ является выражением типа, задающим тип функции, отображающей значения типа T_1 в значения типа T_2 .
3. Других выражений типа нет.

Представление выражения типа дагом

Выражения типа удобно представлять в виде направленного ациклического графа (ДАГа), где общие подтипы не дублируются, а представлены одним узлом. Это экономит память и упрощает сравнение типов.

Как строится:

Есть таблица (хэш-таблица) уже созданных узлов типов.

При создании нового выражения типа:

Разбираем его на компоненты.

Для каждого компонента рекурсивно создаём/находим узел.

Проверяем, есть ли уже узел с такими полями (например, ARRAY размером 10, указывающий на узел ARRAY[5]).

Если есть — используем существующий, если нет — создаём новый.

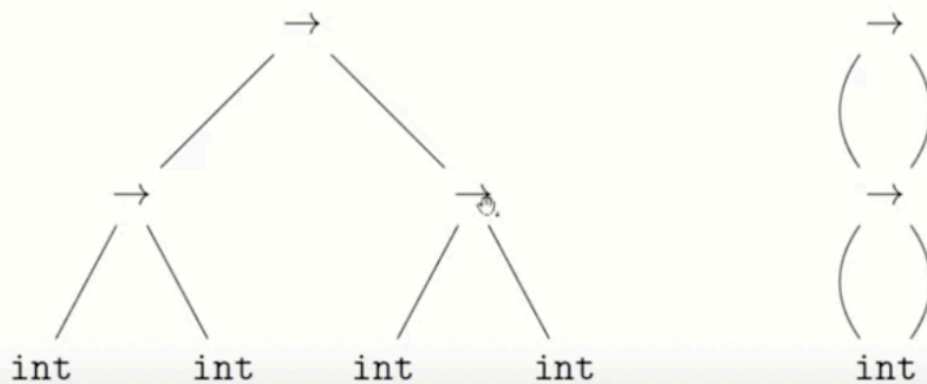
Ключевое: узел типа создаётся единожды.

Представление в виде дерева и дага

Листья — константы, имена и переменные типа, внутренние узлы — конструкторы типа.

Пример.

Функция f вычисляет по любой перестановке τ множества $\{1, \dots, n\}$ квадрат этой перестановки. Тогда f имеет тип $(\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow \text{int})$, задаваемый синтаксическим деревом и дагом:



Представление рекурсивных выражений

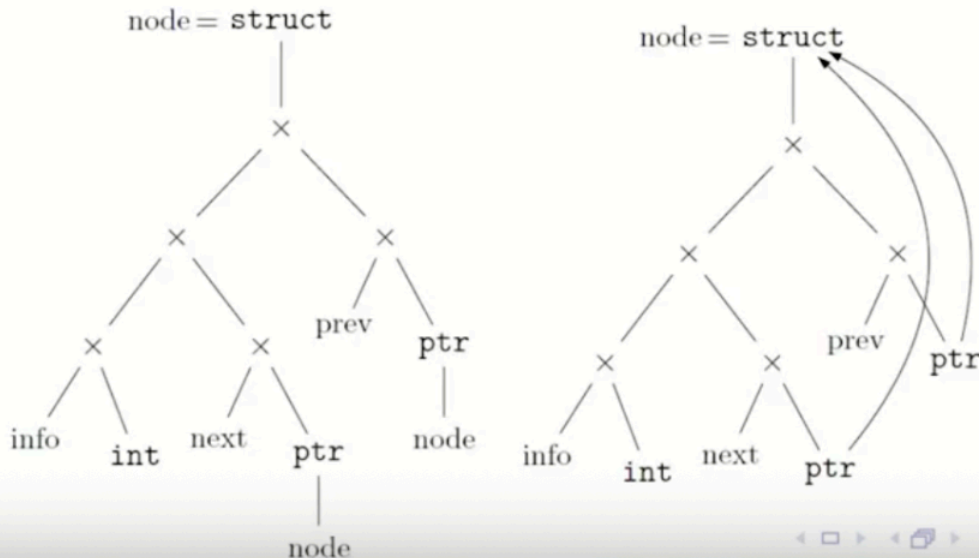
Более сложная ситуация, когда выражение типа является рекурсивным (например, при построении списков).

Пример.

Рассмотрим объявление типа структуры в C для построения двусвязного списка:

```
struct node { int info; struct node *next, *prev; }
```

Соответствующее выражение типа можно представить двумя способами: либо без циклов, но используя имя объявляемого типа в качестве меток листьев, либо вводя циклы.



Эквивалентность выражений типа

Мы хотим понять, одинаковы ли два типа данных в программе. Например, одинаков ли тип "массив чисел" и "список чисел"? Компьютер проверяет это, глядя на их структуру (как они устроены внутри).

Что такое "структурная эквивалентность"?

В типах данных:

array[10] of int (массив из 10 целых чисел)

array[10] of int (другой массив из 10 целых чисел)

Структурно эквивалентны — одинаковое устройство.

Как проверяют эквивалентность? Алгоритм:

Смотрим на корни (верхние узлы) двух типов.

Если метки разные (например, один "массив", другой "запись") → типы разные.

Если метки одинаковые:

Для простых конструкторов (массив, указатель): сравниваем их "детей" (что внутри) рекурсивно.

Для сложных/рекурсивных (запись, класс): сравниваем имена типов, а не структуру.

Рекурсивный тип — это тип, который ссылается сам на себя.

Для рекурсивных типов сравниваются имена, а не структура. Почему?

struct Node и struct ListNode могут быть устроены одинаково (int + указатель), но разные по смыслу.

Иначе можно было бы случайно смешать логически разные типы.

Имя типа становится меткой узла в DAG, и это метка уникальна.

Эквивалентность выражений типа

Определение

Выражения типа называются *структурно эквивалентными*, если представляющие их даги изоморфны.

Следующее наблюдение легко преобразовать в рекурсивный алгоритм проверки изоморфизма дагов: *два дага изоморфны, если их корни имеют одинаковые метки, а подграфы, порожденные соответствующими сыновьями корней, изоморфны. (*)*

*Рекурсивные выражения типа структурно не эквивалентны, если их имена различны. (**)* Это связано с тем, что имя типа появляется в качестве метки узла дага. Поскольку, естественно, одно и то же имя нельзя использовать для обозначения разных типов, структурная эквивалентность рекурсивных выражений типа есть в точности равенство их имен.

Проверка эквивалентности

В свете замечания (**), алгоритм проверки изоморфизма, предложенный в замечании (*), можно уточнить следующим образом: если метки корней проверяемых дагов совпадают, то

- в случае, когда эти метки представляют «потенциально рекурсивный» конструктор (например, запись), проверяется равенство имен дагов;
- если же корни помечены «нерекурсивным» конструктором, то выполняется проверка на изоморфизм дагов, порожденных соответствующими сыновьями.